La Conteneurisation avec Docker et Kubernetes

La conteneurisation est une technologie révolutionnaire qui a transformé la façon dont les applications sont développées, déployées et gérées. Cette présentation vous guidera à travers les concepts clés de la conteneurisation avec Docker et Kubernetes, deux outils puissants qui ont façonné l'industrie du logiciel moderne.

E **by Etienne Koa**

Made with Gamma

Introduction à Docker et à la Technologie des Conteneurs

Bienvenue à cette introduction sur les technologies des conteneurs. Nous allons découvrir ce qu'est Docker, ce qu'est un conteneur, et comment ils se comparent aux machines virtuelles (VM).

Isolation des Services

Pour une meilleure gestion, nous isolons nos services (comme Tomcat, Apache, MySQL) sur différentes machines ou instances, évitant ainsi les interférences causées par le partage des ressources. Traditionnellement, cela se fait via des machines virtuelles, mais cela engendre des coûts élevés en termes de matériel, licences, et maintenance.

Problèmes avec les Machines Virtuelles

Les VM nécessitent chacune un système d'exploitation complet, ce qui augmente les coûts et les temps de démarrage. Elles sont lourdes et complexes à gérer, surtout quand il faut les déplacer ou les distribuer.

Introduction aux Conteneurs

Les conteneurs, contrairement aux VM, permettent d'isoler les services sans nécessiter un système d'exploitation complet. Ils partagent le noyau de l'OS hôte, ce qui les rend plus légers et rapides à démarrer. Un conteneur est essentiellement un processus isolé avec ses propres bibliothèques et binaires.

Docker

Docker est un outil qui simplifie la gestion des conteneurs. Il offre un environnement d'exécution de conteneurs, facilite la création et le déploiement de conteneurs via des images standardisées. Docker a popularisé l'utilisation des conteneurs en rendant la technologie plus accessible et facile à utiliser.

Docker Engine

Docker Engine est le cœur de Docker, un service (ou démon) qui s'exécute sur l'OS hôte. Il permet de créer, gérer et exécuter des conteneurs. Docker Engine utilise des API REST ou des commandes CLI pour interagir avec les conteneurs. Il prend en charge la gestion des images, des réseaux, et des volumes de données, rendant la gestion des conteneurs plus automatisée et simplifiée. Docker Engine a joué un rôle crucial dans la démocratisation des conteneurs grâce à son efficacité et à sa facilité d'utilisation.

Docker Container

Un conteneur Docker est une instance d'une image Docker qui est un paquet comprenant tout le nécessaire pour faire tourner une application : code, runtime, bibliothèques, et dépendances. Les conteneurs sont légers et portables, et peuvent être facilement créés, supprimés, arrêtés ou redémarrés. Les conteneurs partagent le noyau de l'OS hôte mais sont isolés les uns des autres, garantissant que les processus ne se perturbent pas mutuellement.

Made with Gamma

Pour configurer Docker Engine sur une instance EC2 sous Ubuntu 18.04, suivez les étapes ci-dessous :

1. **Lancement de l'Instance EC2 :**
   - Lancez une instance EC2 t2.micro avec les paramètres par défaut et nommez-la "docker engine".
   - Créez un groupe de sécurité permettant tout le trafic depuis votre IP, car Docker utilisera différents ports.
   - Créez une paire de clés et lancez l'instance.
2. **Installation de Docker :**
   - Suivez la documentation officielle de Docker pour télécharger et installer Docker Community Edition (CE) sur Ubuntu.
   - Désinstallez les anciennes versions de Docker (comme docker, docker-engine ou docker.io) si nécessaire.
   - Mettez à jour les dépendances, installez la clé GPG, ajoutez le dépôt Docker et installez Docker.
3. **Configuration Post-Installation :**
   - Vérifiez que le service Docker est en cours d'exécution avec la commande `sudo systemctl status docker`.
   - Par défaut, seul l'utilisateur root peut exécuter des commandes Docker. Pour permettre à un autre utilisateur d'exécuter des commandes Docker, ajoutez-le au groupe Docker en utilisant la commande :

bash

```
sudo usermod -aG docker $USER
```

   - Déconnectez-vous et reconnectez-vous pour que les modifications prennent effet.
1. **Test de l'Installation :**
   - Testez l'installation en exécutant une commande Docker pour créer et exécuter un conteneur simple :

bash

```
docker run hello-world
```

   - Cette commande télécharge une image depuis Docker Hub, crée un conteneur et exécute un script qui affiche un message de bienvenue.
1. **Vérification des Conteneurs :**
   - Vérifiez les conteneurs en cours d'exécution avec `docker ps`.
   - Affichez tous les conteneurs, y compris ceux arrêtés, avec `docker ps -a`.

Docker Community Edition (CE) permet d'exécuter des conteneurs de courte durée (exécutant des scripts simples) et des conteneurs en cours d'exécution continue, comme des services web ou des bases de données.

Dans les prochaines étapes, explorez davantage de commandes Docker et familiarisez-vous avec l'interface en ligne de commande (CLI).

Ce texte devrait être clair et facile à suivre pour quiconque souhaite configurer Docker Engine sur une instance EC2.

Made with Gamma

Comprendre Docker

1

Présentation

Docker est un outil de conteneurisation open source qui permet de packager des applications avec leurs dépendances dans des conteneurs légers et portables. Ces conteneurs peuvent être facilement déployés sur différentes plateformes, garantissant une cohérence et une fiabilité dans les environnements de développement, de test et de production.

2

Configuration du Docker

L'installation et la configuration du moteur Docker sur votre système est une étape cruciale. Cela inclut le téléchargement et l'installation du logiciel, ainsi que la configuration des paramètres système nécessaires pour permettre à Docker de fonctionner correctement.

3

Commandes et Concepts Clés

Docker comporte de nombreuses commandes et concepts fondamentaux, tels que les images, les conteneurs, les réseaux et les volumes. Comprendre ces éléments vous aidera à exploiter pleinement le potentiel de Docker pour vos projets.

Made with Gamma

Résumé sur Docker et Docker Hub

Introduction à Docker Hub

Docker Hub est un site essentiel pour Docker, contenant une vaste collection d'images officielles et personnalisées. On y trouve des images pour divers logiciels comme MongoDB, Postgres, Redis, Ubuntu, etc. Les utilisateurs peuvent créer un compte pour y stocker leurs propres images.

Images et Conteneurs Docker

Les images Docker sont des conteneurs arrêtés et archivés, composées de plusieurs couches de données. Lorsqu'un conteneur est exécuté, il fonctionne directement à partir de ces images. Contrairement aux machines virtuelles, les conteneurs sont liés aux images et ne peuvent pas exister sans elles. Les images sont immuables et seules les couches de lecture-écriture dans les conteneurs sont modifiables.

Registres Docker

Docker Hub n'est pas le seul registre. Il existe d'autres registres comme Google Container Registry (GCR), Amazon Elastic Container Registry (ECR), Nexus Three, JFrog Artifactory, et Docker Trusted Registry (DTR). Les utilisateurs peuvent créer leurs propres solutions de stockage d'images en interne.

Utilisation des Conteneurs

Pour créer un conteneur à partir d'une image, on utilise des commandes comme `docker run`. Les conteneurs peuvent être exécutés en arrière-plan avec l'option détachée (-d). Le mappage de ports est nécessaire pour accéder aux services du conteneur depuis l'extérieur.

Gestion des Conteneurs et Images

Les conteneurs peuvent être démarrés, arrêtés, et supprimés avec les commandes `docker start`, `docker stop`, et `docker rm`. Les images peuvent être supprimées avec `docker rmi`, à condition que les conteneurs associés soient supprimés ou arrêtés. Les commandes `docker exec` et `docker attach` permettent d'exécuter des commandes ou de se connecter aux conteneurs.

Conclusion

Docker et Docker Hub facilitent la gestion et le déploiement d'applications en utilisant des conteneurs légers et modulaires. Les images et conteneurs offrent une solution flexible et efficace pour le développement et l'exploitation de logiciels.

Made with Gamma

Construire des Images Docker

Présentation

La création d'images Docker est une étape essentielle dans le processus de conteneurisation. Un Dockerfile est un fichier texte qui contient les instructions nécessaires pour construire une image Docker, définissant les couches de l'image et les configurations spécifiques à l'application.

Entrée et CMD

Les instructions ENTRYPOINT et CMD dans un Dockerfile jouent un rôle crucial. ENTRYPOINT définit le point d'entrée par défaut d'un conteneur, tandis que CMD spécifie la commande par défaut à exécuter lorsque le conteneur démarre.

Docker Composer

Docker Composer est un outil puissant qui permet de définir et d'exécuter des applications multi-conteneurs. Il vous permet de décrire l'ensemble de votre infrastructure dans un seul fichier YAML, facilitant ainsi le déploiement, la gestion et la mise à l'échelle de vos applications conteneurisées.

Made with Gamma

Résumé de la conférence sur les logs Docker

Introduction

Bienvenue à cette conférence sur les logs Docker, où nous nous concentrerons sur les journaux de vos conteneurs.

Commandes de Base pour les Logs

La commande principale pour visualiser les journaux d'un conteneur est `docker logs`. En fournissant le nom ou l'ID du conteneur, vous pouvez accéder à ses journaux.

Processus de Préparation

Avant d'exécuter un conteneur, il faut d'abord tirer l'image nécessaire avec `docker pull NGINX`, puis lancer le conteneur avec `docker run -P NGINX`. Cette opération mappait automatiquement les ports de l'hôte et du conteneur.

Inspection des Images Docker

Utilisez `docker inspect` pour obtenir des métadonnées détaillées sur une image sous format JSON, comprenant des informations cruciales comme `CMD` et `ENTRYPOINT`.

Exécution et Résultats des Conteneurs

Les conteneurs exécutent des scripts définis par `ENTRYPOINT` et `CMD`. En mode détaché (`-d`), le conteneur s'exécute en arrière-plan, mais vous pouvez visualiser sa sortie avec `docker logs`.

Gestion des Conteneurs

L'exécution sans l'option `-d` affiche les résultats directement à l'écran, mais si vous interrompez le processus (Ctrl+C), le conteneur s'arrête également. Pour visualiser tous les conteneurs, y compris ceux arrêtés, utilisez `docker ps -a`.

Dépannage avec Docker Logs

En cas de problèmes lors de l'exécution d'un conteneur, les logs Docker permettent de diagnostiquer les erreurs. Par exemple, si un conteneur MySQL ne démarre pas, `docker logs` peut indiquer qu'une variable d'environnement nécessaire n'est pas définie.

Exemple de Dépannage

Pour MySQL, il peut être nécessaire de définir une variable d'environnement, par exemple, `-e MYSQL_ROOT_PASSWORD=my_password`, afin de résoudre les erreurs d'initialisation indiquées dans les logs.

Importance des Logs pour le Développement

Les logs sont essentiels pour dépanner les problèmes lors de la construction et de l'exécution de vos propres images Docker. Ils vous permettent de comprendre les erreurs et de les corriger en ajustant les configurations nécessaires.

Conclusion

Comprendre et utiliser les logs Docker est crucial pour le dépannage et la gestion des conteneurs. Avant de passer à la conférence suivante, nettoyez votre environnement en supprimant tous les conteneurs et images inutilisés.

Made with Gamma

Volumes Docker

Les conteneurs Docker sont volatiles et jetables, ce qui signifie qu'ils sont conçus pour être supprimés et recréés à partir d'images. Les modifications apportées à un conteneur doivent être faites via l'image, car les données stockées dans le conteneur disparaissent lorsqu'il est supprimé. Pour gérer les données persistantes, Docker propose deux options : les volumes et les bind mounts.

1. **Volumes Docker** : Les volumes sont des répertoires créés par Docker pour stocker des données persistantes, situés dans `/var/lib/docker/volume` sous Linux. Les données du conteneur sont sauvegardées dans un répertoire spécifique de la machine hôte, ce qui permet de les préserver même après la suppression du conteneur.
2. **Bind Mounts**: Permettent de mapper n'importe quel répertoire de la machine hôte à un répertoire du conteneur. Cela est utile pour injecter des données ou modifier des fichiers en direct depuis la machine hôte, avec les modifications reflétées dans le conteneur.

L'exemple illustré utilise une image MySQL. Le processus implique de :

- Récupérer l'image MySQL.
- Utiliser `docker inspect` pour trouver les détails comme les ports et les répertoires de stockage des données.
- Exécuter le conteneur avec un volume Docker ou un bind mount pour les données persistantes.
- Utiliser `docker logs` pour vérifier les logs du conteneur, utile pour le dépannage.

Enfin, l'importance de nettoyer les conteneurs et les images inutilisés est soulignée pour éviter d'épuiser les ressources.

Made with Gamma

Construire des images

Pour créer des images Docker personnalisées, on part d'images officielles disponibles sur Docker Hub et on y apporte des modifications en fonction des besoins. Ces modifications se font à l'aide d'un fichier Dockerfile, où l'on spécifie les instructions pour construire l'image, y compris les paquets à installer, les fichiers à ajouter, et les configurations nécessaires.

Le Dockerfile contient des instructions simples, comme RUN pour exécuter des commandes, ADD et COPY pour ajouter des fichiers, CMD et ENTRYPOINT pour définir le processus à lancer lors de l'exécution du conteneur, et EXPOSE pour spécifier les ports. Pour créer l'image, on utilise la commande `docker build`.

On peut ensuite tester cette image localement avant de la pousser sur Docker Hub pour la rendre disponible publiquement ou dans un registre privé. L'image doit être correctement nommée (par exemple, inclure le nom d'utilisateur Docker Hub) avant de la pousser avec `docker push`. Une fois publiée, l'image peut être récupérée et exécutée par n'importe qui, n'importe où.

Made with Gamma

CMD ET ENTRYPOINT

Dans cette conférence, l'orateur aborde les instructions ENTRYPOINT et CMD du fichier Docker, car ces deux instructions sont souvent confondues. Il propose un cours séparé pour expliquer ces concepts en détail.

Ensuite, l'orateur passe à trois exemples pratiques de fichiers Docker utilisant CMD et ENTRYPOINT. Dans le premier exemple, un fichier Docker utilise l'instruction CMD pour exécuter la commande echo hello. L'image Docker est construite et exécutée avec succès, affichant le texte "hello".

Dans le deuxième exemple, un fichier Docker utilise l'instruction ENTRYPOINT avec la commande echo, mais sans argument. L'image Docker est également construite et exécutée, mais elle n'affiche rien car l'argument est manquant. L'orateur explique que lorsqu'ENTRYPOINT est utilisé sans argument, l'utilisateur doit le transmettre lors de l'exécution du conteneur.

Enfin, dans le troisième exemple, un fichier Docker utilise à la fois ENTRYPOINT et CMD. L'orateur explique que CMD peut être utilisé pour fournir un argument par défaut, que l'utilisateur peut écraser lors de l'exécution du conteneur. L'image Docker est construite et exécutée avec succès, montrant comment l'argument peut être remplacé ou omis.

En résumé, l'utilisation conjointe de ENTRYPOINT et CMD permet de créer des images Docker flexibles avec des arguments par défaut, offrant ainsi une meilleure gestion des conteneurs.

Made with Gamma

DOCKER COMPOSE

Dans cette conférence, l'orateur présente Docker Compose, un outil distinct de Docker Engine et Docker CLI. Il explique que Docker Compose permet d'exécuter plusieurs conteneurs ensemble, facilitant la communication entre eux. L'utilisation de Docker Compose est particulièrement utile lorsque plusieurs services doivent être déployés et gérés sur le même réseau.

L'orateur donne un aperçu des étapes nécessaires pour utiliser Docker Compose, en soulignant l'écriture d'un fichier docker-compose.yml. Ce fichier contient des instructions pour définir les services (ou conteneurs) à exécuter, ainsi que leurs configurations.

À travers un exemple, l'orateur montre comment Docker Compose simplifie le processus de gestion des conteneurs, en évitant les erreurs courantes liées à l'exécution manuelle de plusieurs commandes Docker. Ils mettent en évidence la facilité d'utilisation de Docker Compose pour lancer et arrêter les conteneurs, ainsi que pour gérer les dépendances entre eux.

Enfin, l'orateur encourage les participants à expérimenter avec Docker Compose en suivant un guide pas à pas pour conteneuriser un projet. Ils soulignent l'importance des fichiers Docker et Docker Compose dans le processus de conteneurisation et mettent en avant l'alternative offerte par Docker Compose pour faciliter le déploiement et la gestion des applications conteneurisées.

Made with Gamma

DOCKER MULTI-STAGE

Dans cette conférence, l'orateur aborde l'importance du fichier Docker multi-étapes pour la construction d'images Docker. Ils expliquent que le processus de construction manuel de l'artefact peut être fastidieux et non automatisé. En examinant un fichier Docker existant, l'orateur met en évidence les défauts de ce processus, soulignant la nécessité de séparer la construction de l'artefact de son incorporation dans l'image finale.

L'orateur propose une approche multi-étapes en utilisant deux images Docker distinctes : une pour la construction de l'artefact et une pour l'application finale. Ils détaillent les étapes pour créer ces images et expliquent comment copier l'artefact de l'image de construction vers l'image finale. L'utilisation de cette méthode permet de réduire la taille de l'image finale et d'automatiser le processus de construction.

En concluant, l'orateur souligne l'importance du fichier Docker multi-étapes dans tout projet nécessitant la construction d'images Docker. Ils encouragent les participants à expérimenter avec ce type de fichier pour optimiser leurs processus de construction d'images Docker.

Made with Gamma

Avantages de Docker

- Isolation des processus sans OS complet
- Rapidité et légèreté
- Réduction des coûts liés aux licences et à la maintenance
- Portabilité et standardisation des applications

Made with Gamma

La Conteneurisation en Pratique

1 Présentation

La conteneurisation apporte de nombreux avantages, tels que la portabilité, l'évolutivité et la facilité de déploiement. En encapsulant les applications dans des conteneurs, vous pouvez les déployer rapidement et de manière cohérente dans différents environnements.

2 Aperçu de l'Image de Base

Le choix de l'image de base appropriée est crucial pour la construction de vos propres images Docker. Ces images de base fournissent un point de départ avec un environnement d'exécution et des dépendances préinstallées, vous permettant de vous concentrer sur l'emballage de votre application.

3 Configuration de Dockerhub

Dockerhub est un référentiel en ligne pour les images Docker, où vous pouvez stocker et partager vos propres images, ainsi qu'accéder à des milliers d'images open source. Configurer un compte Dockerhub vous aidera à automatiser le processus de construction et de déploiement de vos conteneurs.

Made with Gamma

Résumé du projet de conteneurisation

Ce projet se concentre sur la conteneurisation d'applications existantes. Nous examinerons quand et pourquoi utiliser des technologies de conteneur, les avantages concrets, et comment conteneuriser une application multi-niveaux. Actuellement, les services sont souvent exécutés sur des machines virtuelles (VM) ou des environnements cloud, ce qui entraîne des coûts élevés et des gaspillages de ressources. Les déploiements fréquents augmentent également le risque d'erreurs humaines et de désynchronisation entre différents environnements (développement, QA, production).

La conteneurisation offre une solution efficace, réduisant les coûts et les ressources nécessaires grâce à l'absence de système d'exploitation dédié pour chaque application. Les conteneurs améliorent la portabilité et la réutilisabilité des applications, facilitant des déploiements uniformes et rapides dans divers environnements.

Statistiques inspirantes :

- 50% des organisations ont conteneurisé la moitié de leurs applications.
- 29% utilisent des conteneurs en production.
- 78% des gros utilisateurs de cloud utilisent AWS pour leurs conteneurs.
- 81% des équipes DevOps travaillent avec des conteneurs.

Le projet utilisera Docker pour conteneuriser la pile d'applications VProfile, comprenant des services comme nginx, TomCat, MySQL, MemCached, et RabbitMQ. Nous utiliserons Docker Compose et Docker Hub pour gérer et héberger les images. Les étapes incluent :

1. Choisir les bonnes images de base pour chaque service.
2. Écrire des Dockerfiles pour les services nécessitant des personnalisations.
3. Construire et tester les images Docker.
4. Utiliser Docker Compose pour lancer tous les conteneurs ensemble.
5. Héberger les images sur Docker Hub.

Le processus de conteneurisation implique :

- Récupérer le code source depuis le dépôt Git.
- Écrire des Dockerfiles pour nginx, TomCat, et MySQL.
- Utiliser la commande Docker Build pour créer les images personnalisées.
- Tester les conteneurs avec Docker Compose.
- Pousser les images sur Docker Hub.

Pour participer à ce projet, une compréhension de base de Docker et de la manipulation des conteneurs est nécessaire.

Made with Gamma

Résumé et Synthèse du Projet

Le texte présente une procédure détaillée pour la conteneurisation d'un projet à l'aide de Docker et Docker Compose. Voici une synthèse précise et détaillée des étapes clés et des concepts abordés :

1. **Écriture des Fichiers Docker :**
   - Le processus commence par la rédaction des fichiers Docker et docker-compose. Ces fichiers fournissent une vue d'ensemble du projet de conteneurisation.
2. **Collecte des Informations :**
   - La première étape de tout projet est la collecte des informations nécessaires à la construction et au déploiement du projet. Cela inclut la compréhension des étapes de développement, de déploiement et de configuration de l'application en collaboration avec les développeurs.
3. **Création du Fichier docker-compose :**
   - Le fichier docker-compose est créé en mentionnant les différents services utilisés dans le projet (par exemple, base de données, Memcached, RabbitMQ, Tomcat, Nginx).
   - Il faut indiquer l'emplacement des fichiers Docker pour chaque service et décrire comment l'application est déployée dans le Dockerfile.
4. **Utilisation de Docker Hub :**
   - On peut utiliser des images préexistantes disponibles sur Docker Hub, comme Maven ou OpenJDK, pour simplifier certaines étapes.
   - L'étape ADD permet de monter le dossier du référentiel actuel dans l'image du conteneur.
5. **Construction et Déploiement des Images :**
   - Les artefacts de construction doivent être séparés des images d'application. On copie l'artefact de l'image de construction vers l'image d'application.
   - Pour les bases de données, il est courant d'utiliser des images MySQL ou Redis et de configurer les schémas, noms d'utilisateur et noms de base de données.
6. **Configuration du docker-compose :**
   - Le fichier docker-compose configure les ports et les variables d'environnement nécessaires au fonctionnement des conteneurs.
7. **Processus de Test et de Déploiement :**
   - Une fois les images conteneurisées, elles peuvent être testées et poussées sur Docker Hub si elles sont valides.
   - Le processus final consiste à cloner le code source, à utiliser docker-compose pour construire et exécuter les conteneurs.
8. **Résumé des Étapes Suivies :**
   - Connaître les étapes pour installer la pile de services sur des machines virtuelles ou localement.
   - Trouver la bonne image de base sur Docker Hub.
   - Écrire des Dockerfile pour personnaliser les images.
   - Utiliser docker-compose pour lancer et tester plusieurs conteneurs.
   - Héberger les images validées sur Docker Hub.
9. **Conclusion :**
   - Ce projet a permis de maîtriser la conteneurisation des applications, en passant par la collecte d'informations, la configuration des fichiers Docker, le test et le déploiement des conteneurs.

Ce texte offre une vue complète du processus de conteneurisation, illustrant chaque étape nécessaire pour construire, tester et déployer des applications conteneurisées avec Docker et Docker Compose.

Introduction aux Microservices et à la Conteneurisation

- **Microservices** : Architecture logicielle où l'application est divisée en services indépendants.
- **Conteneurisation** : Processus d'encapsulation d'un logiciel et de ses dépendances dans un conteneur pour assurer une exécution cohérente.

Avantages des Microservices

- Permettent un développement et une livraison rapides et efficaces des fonctionnalités.
- DevOps doivent maîtriser la conteneurisation des microservices.

Exemples et Structure de l'Application

- **Application exemple** : e-commerce avec 1234 services.
- **Services inclus** : NGINX (passerelle API), client Angular, API NodeJS, base de données MongoDB, API Java avec base de données SQL.

Organisation du Code Source

- **Mono repo** : Tout le code source des microservices dans un seul dépôt GitHub nommé Emartapp.
- **Structure** : Inclut des sous-dossiers pour chaque service et les fichiers de configuration nécessaires.

Conteneurisation avec Docker

1. **Fichier Docker pour chaque service** :
   - **Client Angular** : Image basée sur Node, construction de l'application et déploiement sur NGINX.
   - **API NodeJS** : Image Node, construction et exécution du service.
   - **API Java** : Image JDK avec Maven, construction de l'application et exécution.
   - **NGINX** : Utilisation de l'image officielle avec une configuration personnalisée.
2. **Docker Compose** :
   - **Services** : Client, API Node, API Java, NGINX, MongoDB, MySQL.
   - **Définition des dépendances** : Assurer que les services démarrent dans le bon ordre.
   - **Mapping des ports** et configuration des volumes.

Processus de Construction et Hébergement

- **Construction** : Instructions pour installer les dépendances et compiler le code.
- **Hébergement** : Placement des artefacts construits dans le conteneur approprié (e.g., fichiers Angular dans NGINX).

Bonnes Pratiques et Dépannage

- **Collaborer avec les développeurs** pour comprendre les étapes de construction et les méthodes d'hébergement.
- **Patience et rigueur** : S'attendre à des erreurs et les résoudre en vérifiant les journaux et configurations.
- **Redémarrage automatique** des conteneurs en cas d'échec, surtout pour les bases de données.

Conclusion

- **Fichiers Docker et Docker Compose** : Éléments clés pour la conteneurisation.
- **Documentation et Support** : Utiliser la documentation officielle et les forums pour toute question ou problème.

Résumé des étapes

1. **Cloner le dépôt** depuis GitHub.
2. **Ouvrir dans Visual Studio ou un autre IDE**.
3. **Examiner et comprendre les fichiers Docker et Docker Compose**.
4. **Configurer et lancer les services avec Docker Compose**.

Cette conférence fournit une vue d'ensemble pratique et détaillée pour conteneuriser des microservices en utilisant Docker et Docker Compose, en mettant l'accent sur la collaboration avec les développeurs et la compréhension des processus de construction et d'hébergement des applications.

Made with Gamma

Kubernetes : La Gestion des Conteneurs à Grande Échelle

1 Présentation

Kubernetes est un système open source pour l'orchestration des conteneurs, permettant de déployer, mettre à l'échelle et gérer des applications conteneurisées de manière automatisée. Il offre des fonctionnalités avancées de gestion de clusters, de réplication, de mise à l'échelle et de haute disponibilité.

2 Configuration de Minikube

Minikube est un outil permettant de configurer facilement un environnement Kubernetes local sur votre machine de développement. Il vous aidera à vous familiariser avec les concepts de base de Kubernetes et à tester vos applications conteneurisées avant de les déployer sur un cluster de production.

3 Objets et Documentation

Kubernetes fournit de nombreux objets tels que Pods, Services, Déploiements et Volumes. Comprendre ces objets et leur rôle dans l'orchestration des conteneurs est essentiel pour tirer le meilleur parti de Kubernetes. La documentation officielle de Kubernetes est une ressource inestimable pour approfondir vos connaissances.

Made with Gamma

Déploiement d'Applications sur Kubernetes

Secrets Kubernetes

Les secrets Kubernetes permettent de stocker de manière sécurisée des informations sensibles telles que les mots de passe, les clés API et les certificats. Cela garantit que ces données critiques ne sont pas exposées dans vos fichiers de configuration.

Définition des Déploiements

Les fichiers de déploiement Kubernetes définissent la façon dont vos applications conteneurisées doivent être déployées, y compris le nombre de réplicas, les images à utiliser et les paramètres de configuration. Ces fichiers de déploiement sont essentiels pour assurer un déploiement cohérent et reproductible de vos applications.

Services et Ingress

Les services Kubernetes fournissent un moyen stable d'accéder à vos applications conteneurisées, en répartissant le trafic entre les pods. L'Ingress permet de configurer des règles d'acheminement HTTP/HTTPS sophistiquées pour exposer vos services de manière sécurisée et cohérente.

Gestion des Volumes

La gestion des volumes est cruciale pour la persistance des données dans un environnement Kubernetes. Les volumes Kubernetes permettent de monter des stockages externes tels que des systèmes de fichiers, des bases de données ou des partages réseau dans vos pods, garantissant la durabilité des données.

Made with Gamma

De la Conteneurisation à la Conteneurisation à grande échelle

Conteneurisation de Microservices

Les architectures de microservices deviennent de plus en plus répandues, et la conteneurisation joue un rôle essentiel dans leur mise en œuvre. En encapsulant chaque service dans un conteneur, vous bénéficiez d'une plus grande flexibilité, d'une meilleure évolutivité et d'une plus grande résilience.

Déploiement dans le Cloud

Le déploiement d'applications conteneurisées sur des plateformes cloud comme AWS, Azure ou Google Cloud Platform offre de nombreux avantages, tels que l'évolutivité, la haute disponibilité et la facilité de gestion. Les outils d'orchestration de conteneurs comme Kubernetes s'intègrent parfaitement à ces environnements cloud.

Automatisation du Cycle de Vie

L'automatisation du cycle de vie des applications conteneurisées, de la construction à la mise en production, est essentielle pour garantir des déploiements rapides, fiables et cohérents. Des outils comme Jenkins, CircleCI ou GitLab CI peuvent être intégrés pour automatiser entièrement le processus de déploiement.

Mise à l'Échelle et Résilience

Grâce à la conteneurisation et à l'orchestration par Kubernetes, vous pouvez facilement mettre à l'échelle vos applications en fonction de la demande, en ajoutant ou en supprimant des instances de conteneurs. Cette élasticité, combinée à la résilience apportée par Kubernetes, vous permet de gérer efficacement les pics de charge et les défaillances.

Made with Gamma

L'Avenir de la Conteneurisation

1

Adoption Croissante

La conteneurisation et Kubernetes continuent de gagner en popularité dans l'industrie du logiciel, devenant de facto des standards pour le déploiement d'applications modernes.

2

Intégration avec le Cloud

L'intégration transparente des technologies de conteneurisation avec les principales plateformes cloud favorise leur adoption à grande échelle, permettant aux entreprises de tirer parti des avantages du cloud.

3

Innovations Technologiques

Des innovations continues dans les domaines de la sécurité, de la gestion des données et de l'observabilité des conteneurs vont renforcer les capacités de la conteneurisation et de Kubernetes.

4

Adoption dans de Nouveaux Domaines

Au-delà des applications web traditionnelles, la conteneurisation s'étend à des domaines tels que l'Internet des objets, l'apprentissage machine et l'edge computing, ouvrant de nouvelles perspectives passionnantes.

Made with Gamma